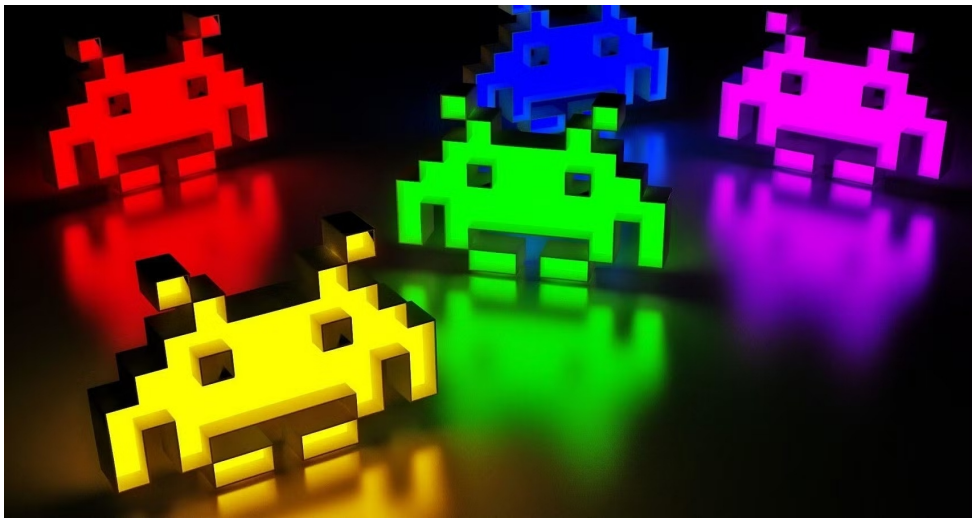
Space Invaders

Contrôle par Détection d'Objet Coloré (HSV)



DANFA Malicoumba
RAJAGOPAL Sarnuja
MAOUCHE Amirouche
SOUICI Ines

Encadrant : M. Florian VALADE

Table des matières

1 Architecture du système **2**
1.1 Flux de données 2

2 Approche de vision par ordinateur **3**
2.1 Choix de l'espace colorimétrique HSV 3
2.2 Plage de couleur verte 3
2.3 Nettoyage du masque par morphologie 3
2.4 Détection du contour et calcul du centroïde 4

3 Logique de décision **4**

4 Interface visuelle **5**
4.1 Fenêtre principale — flux webcam annoté 5
4.2 Fenêtre de debug — masque binaire 5

5 Implémentation technique **5**
5.1 Structure du script 5
5.2 Gestion asynchrone avec asyncio 6
5.3 Effet miroir 6
5.4 Dépendances 6

6 Instructions de lancement **6**

7 Difficultés rencontrées et solutions **7**

8 Pistes d'amélioration **7**

Introduction et Objectif du projet

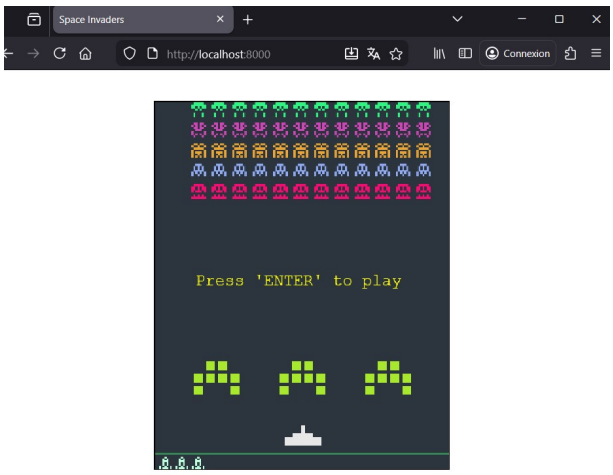
L'objectif de ce projet est de contrôler le jeu **Space Invaders** en temps réel par vision par ordinateur, **sans utiliser le clavier**. Le script Python capture le flux de la webcam, détecte la position d'un **objet vert** dans l'image et traduit sa position en commandes de jeu envoyées via WebSocket.

Cette approche correspond au **Niveau 3** des consignes : joystick physique par détection de couleur. Elle repose exclusivement sur **OpenCV** et **NumPy**, sans dépendance à un modèle de deep learning, ce qui la rend légère et très rapide.

1 Architecture du système

Le système repose sur trois composants indépendants qui communiquent en temps réel :

- **Serveur HTTP** (Python) : sert les fichiers du jeu sur `http://localhost:8000`
- **Serveur WebSocket** (Node.js) : pont entre le script Python et le navigateur, sur le port `8765`
- **Module de contrôle** (`cv_controller.py`) : analyse la webcam et envoie les commandes



1.1 Flux de données

`Webcam` ⟶ `Flip miroir` ⟶ `Conversion HSV` ⟶ `Masque couleur` ⟶ `Détection contours` ⟶ `Centroïde` ⟶ `Règles de décision` ⟶ `WebSocket` ⟶ `Jeu`

2 Approche de vision par ordinateur

2.1 Choix de l'espace colorimétrique HSV

La détection de couleur est réalisée dans l'espace **HSV** (Hue, Saturation, Value) plutôt qu'en RGB. Ce choix est fondamental : en RGB, la couleur d'un objet varie fortement selon l'éclairage, car les trois canaux R, G et B sont tous affectés. En HSV, la **teinte** (Hue) est séparée de la **luminosité** (Value), ce qui rend la détection beaucoup plus robuste aux changements de lumière ambiante.

La conversion est effectuée avec :

```
image_hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
```

2.2 Plage de couleur verte

La couleur verte est définie par les bornes HSV suivantes :

Paramètre	Minimum	Maximum
Teinte (H)	35	85
Saturation (S)	50	255
Valeur / Luminosité (V)	50	255

Ces valeurs couvrent un spectre de vert assez large (du vert-jaune au vert foncé) tout en excluant les verts très désaturés (gris) ou très sombres (noir), qui seraient des faux positifs fréquents.

```
VERT_MIN = np.array([35, 50, 50])
VERT_MAX = np.array([85, 255, 255])
masque = cv2.inRange(image_hsv, VERT_MIN, VERT_MAX)
```

2.3 Nettoyage du masque par morphologie

Le masque binaire brut contient souvent du bruit (pixels isolés, reflets). Deux opérations morphologiques sont appliquées pour l'éliminer :

- **Érosion** (`cv2.erode`) : supprime les petits artefacts isolés en rétrécissant les zones blanches
- **Dilatation** (`cv2.dilate`) : restaure la taille originale de l'objet après érosion

```
masque = cv2.erode(masque,  None, iterations=2)
masque = cv2.dilate(masque, None, iterations=2)
```

Cette combinaison (érosion puis dilatation) est appelée **ouverture morphologique**. Elle supprime efficacement le bruit tout en préservant la forme de l'objet principal.

2.4 Détection du contour et calcul du centroïde

Les contours des zones blanches du masque sont extraits avec `cv2.findContours`. Parmi tous les contours détectés, le **plus grand** est conservé, il correspond à l'objet tenu en main.

Un filtre de taille minimale élimine les résidus trop petits :

```
TAILLE_MIN = 500  # pixels
plus_grand = max(contours, key=cv2.contourArea)
if cv2.contourArea(plus_grand) < TAILLE_MIN:
    return None
```

La position précise de l'objet est calculée via les **moments géométriques** du contour, qui donnent le centroïde (barycentre de la forme) :

```
moments = cv2.moments(plus_grand)
cx = int(moments["m10"] / moments["m00"])   # coordonnee x
cy = int(moments["m01"] / moments["m00"])   # coordonnee y
```

Les coordonnées sont ensuite **normalisées** entre 0.0 et 1.0 en les divisant par les dimensions de l'image, ce qui rend la logique de décision indépendante de la résolution de la webcam.

3 Logique de décision

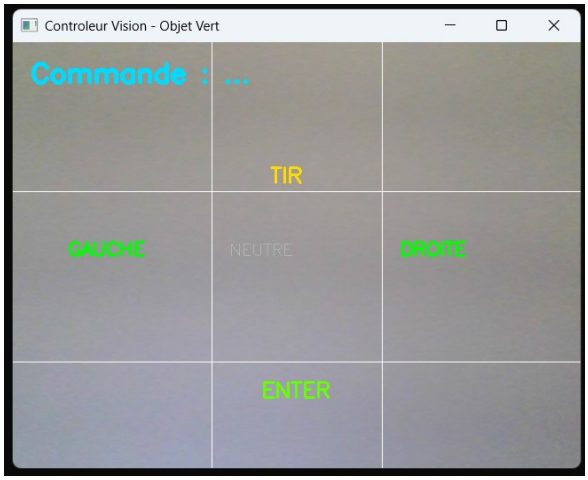
L'image est divisée en zones fonctionnelles. La position normalisée du centroïde `(x, y)` détermine la commande à envoyer, selon un ordre de **priorités strictes** :

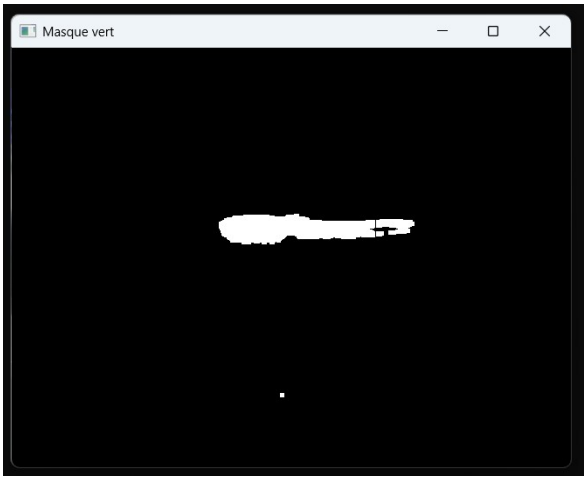
Priorité	Condition	Commande	Action
1	$y > 0.75$ (objet en bas)	`ENTER`	Démarrer / valider
2	$y < 0.35$ (objet en haut)	`FIRE`	Tirer
3	$x < 0.35$ (objet à gauche)	`LEFT`	Déplacer à gauche
4	$x > 0.65$ (objet à droite)	`RIGHT`	Déplacer à droite
5	Zone centrale	`None`	Aucune action

La gestion ENTER en priorité absolue évite qu'un mouvement de démarrage soit interprété comme un tir. La zone neutre centrale (entre 35% et 65%) sert de zone tampon pour éviter les faux déclenchements involontaires.

4 Interface visuelle

Deux fenêtres OpenCV sont affichées simultanément :





4.1 Fenêtre principale — flux webcam annoté

La fenêtre "Controleur Vision - Objet Vert" affiche le flux webcam en temps réel avec :

- Les **lignes de séparation** des zones de contrôle (blanches)
- Les **labels** des zones : TIR, GAUCHE, NEUTRE, DROITE, ENTER
- Le **contour vert** dessiné autour de l'objet détecté
- Le **centroïde** (point vert plein) marquant le centre de l'objet
- La **commande en cours** affichée en haut à gauche en jaune

4.2 Fenêtre de debug — masque binaire

La fenêtre "Masque vert" affiche le masque HSV en noir et blanc, permettant de visualiser exactement ce que le programme "voit" en vert. Cette fenêtre est précieuse pour calibrer les seuils HSV en fonction de l'éclairage de la pièce.

5 Implémentation technique

5.1 Structure du script

Le code est organisé en trois fonctions bien séparées :

- `analyser_image(image)` : toute la logique de vision et de décision
- `dessiner_zones(image)` : rendu de l'interface visuelle
- `controleur_vision()` : boucle principale asyncio, webcam et WebSocket

5.2 Gestion asynchrone avec asyncio

Le script utilise `asyncio` en boucle principale, ce qui permet d'intégrer nativement les appels WebSocket **non-bloquants** sans thread supplémentaire. Une pause `await asyncio.sleep(0.01)` est insérée à chaque itération pour éviter de saturer le processeur tout en maintenant une fluidité suffisante.

```
async def controleur_vision():
    async with websockets.connect(ADRESSE_SERVEUR) as
       connexion:
        while True:
            succes, image = webcam.read()
            image = cv2.flip(image, 1)          # miroir
               horizontal
            dessiner_zones(image)
            commande = analyser_image(image)
            if commande:
                await connexion.send(commande)  # envoi non-
                   bloquant
            await asyncio.sleep(0.01)           # pause pour
               le processeur
```

5.3 Effet miroir

L'image est retournée horizontalement avec `cv2.flip(image, 1)` avant tout traitement. Cela rend le contrôle intuitif : déplacer l'objet vers la droite dans la vraie vie déplace le vaisseau vers la droite à l'écran.

5.4 Dépendances

Le script déclare ses dépendances directement en en-tête via le format `uv inline script`, ce qui permet de le lancer sans créer d'environnement virtuel :

```
# /// script
# dependencies = ["websockets", "opencv-python"]
# ///
```

Note : `numpy` est utilisé dans le script via `import numpy as np` mais n'est pas déclaré explicitement dans le bloc `uv` car il est une dépendance implicite d'opencv-python.

6 Instructions de lancement

Terminal 1 - Serveur HTTP du jeu :

```
cd space-invaders
python -m http.server 8000
```

Terminal 2 - Serveur WebSocket :

```
cd space-invaders
node server.js
```

Terminal 3 - Module de contrôle CV :

```
cd space-invaders
python cv_controller.py
```

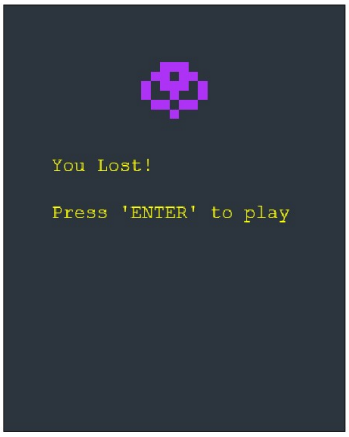
Ouvrir http://localhost:8000 dans le navigateur, tenir un objet vert devant la webcam et le déplacer pour contrôler le vaisseau.

7 Difficultés rencontrées et solutions

- **Fluidité du contrôle** : l'envoi immédiat de la commande dès détection assure une réactivité maximale pour les mouvements latéraux.
- **Effet miroir** : sans le `cv2.flip`, les directions gauche et droite sont inversées, rendant le jeu injouable.

8 Pistes d'amélioration

- **Multi-couleurs** : détecter plusieurs objets de couleurs différentes pour des contrôles distincts (ex. rouge = FIRE, vert = déplacement).
- **Niveau 4** : capturer directement l'état du jeu depuis l'écran avec `mss` et construire un agent par apprentissage par renforcement .

Conclusion

Ce projet démontre qu'une approche de vision par ordinateur classique, sans deep learning, suffit pour créer un contrôleur de jeu fluide et réactif. Le seuillage HSV combiné à l'analyse de contours OpenCV offre un excellent rapport simplicité / performance, avec une latence quasi nulle et une robustesse satisfaisante aux conditions d'éclairage courantes.